## RAPPORT DE TRAVAUX PRATIQUES

# TP 5 — Application Web Java EE

*Architecture MVC2 avec Hibernate & Filtres de Sécurité*

| | |
|---|---|
| **Module** | Développement Web Java EE |
| **TP n°** | TP 5 — MVC2 + Hibernate |
| **Technologies** | Java EE, Servlet, JSP, Hibernate, H2, Maven |
| **Année** | 2024 / 2025 |

# 1. Introduction

Ce travail pratique a pour objectif de concevoir et développer une application web Java EE de gestion de produits en suivant le patron de conception MVC2 (Model-View-Controller, type 2). L'application intègre Hibernate comme couche ORM pour la persistance des données, une base H2 en mémoire, ainsi qu'un système d'authentification et de contrôle d'accès basé sur les rôles.

L'application offre les fonctionnalités suivantes :

- Authentification des utilisateurs (login / logout) avec gestion de session
- Gestion complète des produits : ajout, modification, suppression, recherche et listage
- Contrôle des accès basé sur deux rôles : ADMIN (accès complet) et USER (lecture seule)
- Filtres Servlet pour la sécurité (authentification et gestion des rôles)
- Initialisation automatique de la base de données au démarrage

# 2. Architecture de l'Application

## 2.1 Structure du projet

Le projet suit la structure Maven standard pour une application web Java EE. L'arborescence est organisée en couches bien définies :

```
GestionProduitsMVC2/
├── pom.xml                          (dépendances Maven)
└── src/main/
    ├── java/
    │   ├── models/        Produit.java
    │   ├── dao/           ProduitDAO.java, ProduitDAOImpl.java
    │   ├── services/      ProduitService.java, ProduitServiceImpl.java
    │   ├── controllers/   LoginController.java, ProduitController.java
    │   ├── filters/       AuthFilter.java, RoleFilter.java
    │   └── utils/         HibernateUtil.java, UserStore.java,
AppInitListener.java
    ├── resources/     hibernate.cfg.xml
    └── webapp/
        ├── index.jsp      (redirection vers /login)
        └── WEB-INF/
            ├── web.xml
            └── views/  login.jsp, index.jsp, 404.jsp, error.jsp
```

## 2.2 Schéma de l'architecture MVC2

L'architecture MVC2 sépare clairement les responsabilités entre les différentes couches de l'application :

| Couche | Package | Classes | Rôle |
|---|---|---|---|
| Modèle | models | Produit | Entité JPA représentant un produit en base |
| DAO | dao | ProduitDAO(Impl) | Accès direct à la base via Hibernate |
| Service | services | ProduitService(Impl) | Logique métier et point d'entrée du contrôleur |
| Contrôleur | controllers | ProduitController, LoginController | Dispatch des requêtes HTTP, orchestration |
| Vue | WEB-INF/views | *.jsp | Rendu HTML avec JSTL |
| Filtre | filters | AuthFilter, RoleFilter | Sécurisation transversale des requêtes |

| | | | |
|---|---|---|---|
| Utilitaires | utils | HibernateUtil, UserStore, AppInitListener | Outils techniques transversaux |

# 3. Explication du Code

Ce chapitre décrit en détail chaque composant du projet, son rôle et les choix de conception adoptés.

## 3.1 La couche Modèle — Produit.java

La classe Produit est l'entité centrale de l'application. Elle est annotée avec les annotations JPA (Java Persistence API) pour permettre à Hibernate de la mapper automatiquement sur la table produits de la base de données.

```
@Entity
@Table(name = "produits")
public class Produit {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_produit")
    private Long idProduit;

    @Column(name = "nom", nullable = false)
    private String nom;

    @Column(name = "prix", nullable = false)
    private Double prix;
}
```

Annotations utilisées :

- @Entity : indique à Hibernate que cette classe est une entité persistante
- @Table(name="produits") : précise le nom de la table en base
- @Id + @GeneratedValue : définit la clé primaire générée automatiquement (auto-increment)
- @Column : mappe l'attribut sur une colonne précise, avec contraintes (nullable=false)

## 3.2 La couche DAO (Data Access Object)

Le pattern DAO sépare la logique d'accès aux données du reste de l'application. On définit d'abord une interface ProduitDAO, puis une implémentation ProduitDAOImpl qui utilise Hibernate.

### Interface ProduitDAO

```
public interface ProduitDAO {
    void addProduit(Produit p);
```

```
    void deleteProduit(Long id);
    Produit getProduitById(Long id);
    List<Produit> getAllProduits();
    void updateProduit(Produit p);
}
```

### Implémentation ProduitDAOImpl

Chaque méthode de modification (add, update, delete) ouvre une Session Hibernate, démarre une transaction, exécute l'opération, puis valide ou annule (rollback) en cas d'erreur. La lecture (get, list) se fait sans transaction explicite.

```
@Override
public void addProduit(Produit p) {
    Transaction tx = null;
    try (Session session = HibernateUtil.getSessionFactory().openSession())
{
        tx = session.beginTransaction();
        session.save(p);
        tx.commit();
    } catch (Exception e) {
        if (tx != null) tx.rollback();
        throw e;
    }
}
```

Le try-with-resources (try (Session session = ...)) assure la fermeture automatique de la session, même en cas d'exception, ce qui évite les fuites de ressources.

## 3.3 La couche Service

La couche service sert d'intermédiaire entre les contrôleurs et le DAO. Elle permet d'isoler la logique métier et d'abstraire l'accès aux données. Dans ce TP, la logique est simple, mais la couche reste essentielle pour la séparation des préoccupations.

Le ProduitServiceImpl utilise le pattern Singleton pour garantir qu'une seule instance du service existe pendant toute la durée de vie de l'application :

```
public class ProduitServiceImpl implements ProduitService {

    private static ProduitServiceImpl instance;
    private final ProduitDAO dao;

    private ProduitServiceImpl() {
        dao = new ProduitDAOImpl();
    }

    public static synchronized ProduitServiceImpl getInstance() {
        if (instance == null) {
```

```
            instance = new ProduitServiceImpl();
        }
        return instance;
    }
}
```

Le mot-clé synchronized garantit la sécurité en cas d'accès concurrent (multi-threading dans un serveur d'applications).

## 3.4 Les Contrôleurs MVC2

Le contrôleur est le cœur de l'architecture MVC2. Contrairement à MVC1 où les JSP font office de contrôleurs, ici les Servlets Java prennent en charge le dispatching des requêtes.

### LoginController.java

Ce contrôleur gère l'authentification. Il intercept deux types de requêtes :

- GET /login : affiche le formulaire de connexion (ou exécute la déconnexion si action=logout)
- POST /login : traite les identifiants, crée la session HTTP en cas de succès, ou renvoie une erreur

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp) {
    String login    = req.getParameter("login");
    String password = req.getParameter("password");

    if (UserStore.isValidUser(login, password)) {
        HttpSession session = req.getSession();
        session.setAttribute("user", login);
        session.setAttribute("role", UserStore.getRole(login));
        resp.sendRedirect(req.getContextPath() + "/produits");
    } else {
        req.setAttribute("error", "Login ou mot de passe incorrect");
        req.getRequestDispatcher("/WEB-INF/views/login.jsp").forward(req,
resp);
    }
}
```

### ProduitController.java

Ce contrôleur centralise toutes les opérations CRUD sur les produits. Il utilise le paramètre HTTP action pour dispatcher vers la bonne méthode interne :

| Méthode HTTP | Action | Handler | Description |
|---|---|---|---|
| GET | (aucune) | showList() | Affiche la liste de tous les produits |
| GET | search | handleSearch() | Recherche par identifiant (idProduit) |
| GET | edit | handleEdit() | Pré-remplit le formulaire pour modification |
| GET | delete | handleDelete() | Supprime un produit et redirige |
| POST | add | handleAdd() | Crée un nouveau produit |
| POST | update | handleUpdate() | Met à jour un produit existant |

## 3.5 Les Filtres de Sécurité

Les filtres Servlet sont des composants qui interceptent les requêtes HTTP avant qu'elles n'atteignent le contrôleur cible. Ce TP en utilise deux.

### AuthFilter.java — Filtre d'Authentification

Ce filtre est appliqué à toutes les URL (/*). Son rôle est de vérifier qu'un utilisateur est connecté (session valide) avant de donner accès à toute ressource protégée. Si la session est invalide et que la requête ne vise pas la page de login, le filtre redirige vers /login.

```
boolean isLoggedIn  = (session != null && session.getAttribute("user") !=
null);
boolean isLoginPage = uri.endsWith("/login");

if (isLoggedIn || isLoginPage) {
    chain.doFilter(req, res);  // continuer la chaîne
} else {
    response.sendRedirect(request.getContextPath() + "/login");
}
```

### RoleFilter.java — Filtre de Contrôle des Rôles

Ce filtre, appliqué uniquement à l'URL /produits, vérifie que l'utilisateur possède le rôle ADMIN pour les actions d'écriture (add, update, delete, edit). Les actions de lecture (list, search) sont accessibles à tous les utilisateurs connectés.

```
private static final Set<String> ADMIN_ACTIONS =
    new HashSet<>(Arrays.asList("add", "update", "delete", "edit"));

if (action == null || !ADMIN_ACTIONS.contains(action)) {
    chain.doFilter(req, res);  // lecture autorisée pour tous
    return;
}

// Vérification du rôle ADMIN
String role = (String) session.getAttribute("role");
if ("ADMIN".equals(role)) {
    chain.doFilter(req, res);
} else {
    response.sendError(SC_FORBIDDEN, "Accès réservé à l'administrateur");
}
```

## 3.6 Les Utilitaires

### HibernateUtil.java

Cette classe fournit un point d'accès unique à la SessionFactory Hibernate. La SessionFactory est créée une seule fois au chargement de la classe (via un bloc static), car sa création est coûteuse. Elle est ensuite réutilisée pour toutes les opérations en base.

### UserStore.java

Cette classe simule un annuaire d'utilisateurs en mémoire via deux HashMap : l'une pour les couples login/mot de passe, l'autre pour les rôles. Dans un contexte de production, ces données seraient stockées en base ou dans un serveur LDAP.

```
// Comptes disponibles :
// admin / admin123  → rôle ADMIN (accès complet)
// user1 / pass1     → rôle USER  (lecture seule)
```

### AppInitListener.java

Ce listener ServletContext est annoté avec @WebListener, ce qui lui permet d'être détecté automatiquement au démarrage du serveur. Il initialise la SessionFactory Hibernate et insère trois produits de test dans la base de données. À l'arrêt du serveur, il ferme la SessionFactory.

## 3.7 Les Vues JSP

### login.jsp

Vue minimaliste affichant un formulaire d'authentification (POST vers /login). En cas d'erreur, le message est affiché via la variable EL ${error} positionnée par le contrôleur.

### index.jsp (dans WEB-INF/views)

C'est la vue principale de gestion des produits. Elle utilise la JSTL (JavaServer Pages Standard Tag Library) pour :

- Afficher conditionnellement le formulaire d'ajout/modification uniquement si le rôle est ADMIN (balise <c:if>)
- Basculer entre les modes ajout et modification selon la présence de l'attribut produitEdit (<c:choose>)
- Parcourir la liste de produits avec <c:forEach>
- Afficher les boutons Modifier et Supprimer uniquement aux ADMIN

### 404.jsp et error.jsp

Pages d'erreur déclarées dans web.xml pour les codes HTTP 404, 403 et 500. La page error.jsp affiche également le message d'exception via ${pageContext.exception.message}.

## 3.8 Configuration — hibernate.cfg.xml et web.xml

### hibernate.cfg.xml

Ce fichier configure la connexion à la base H2 en mémoire et le comportement de Hibernate :

- jdbc:h2:mem:produitsdb;DB_CLOSE_DELAY=-1 : base en mémoire, données conservées tant que la JVM tourne
- hbm2ddl.auto=create-drop : les tables sont créées au démarrage et supprimées à l'arrêt
- show_sql=true : affiche les requêtes SQL dans la console (utile pour le débogage)

### web.xml

Le descripteur de déploiement définit les mappings de Servlets et Filtres. Les contrôleurs sont enregistrés manuellement avec leurs URL patterns (/login et /produits). L'AuthFilter couvre toutes les URLs (/*) et le RoleFilter est limité à /produits.

# 4. Étapes d'Implémentation

Ce chapitre décrit la démarche de réalisation du TP, étape par étape, depuis la création du projet jusqu'aux tests finaux.

## Étape 1 — Création du projet Maven

La première étape consiste à créer un projet Maven de type webapp. Le fichier pom.xml est configuré avec les dépendances nécessaires :

- javax.servlet-api (4.0.1) et javax.servlet.jsp-api (2.3.3) en scope provided (fournies par Tomcat)
- jstl (1.2) pour les tags JSTL dans les JSP
- hibernate-core (5.6.15.Final) pour l'ORM
- h2 (2.2.224) comme base de données embarquée, sans installation

**Configuration Maven**

GroupId : ma.tp5
ArtifactId : GestionProduitsMVC2
Packaging : war
Java : 11

## Étape 2 — Modélisation de l'entité Produit

La classe Produit est créée dans le package models avec les annotations JPA. Les attributs retenus sont : idProduit (clé primaire auto-générée), nom (obligatoire), description (facultatif) et prix (obligatoire). Les getters et setters sont générés.

## Étape 3 — Configuration de Hibernate

Le fichier hibernate.cfg.xml est créé dans src/main/resources. On choisit H2 en mémoire pour simplifier le déploiement (aucune installation de base de données requise). La classe utilitaire HibernateUtil est implémentée avec un bloc static pour initialiser la SessionFactory une seule fois.

## Étape 4 — Implémentation de la couche DAO

L'interface ProduitDAO est définie avec les cinq méthodes CRUD. L'implémentation ProduitDAOImpl est réalisée en utilisant les Sessions Hibernate avec gestion des transactions

et rollback en cas d'erreur. Chaque méthode utilise le try-with-resources pour garantir la fermeture automatique des sessions.

## Étape 5 — Implémentation de la couche Service

L'interface ProduitService est définie puis implémentée par ProduitServiceImpl. Le pattern Singleton est appliqué avec synchronisation pour assurer la thread-safety. Les méthodes délèguent directement au DAO.

## Étape 6 — Mise en place de l'authentification

La classe UserStore est créée pour gérer les comptes en mémoire. Deux comptes sont configurés : admin (rôle ADMIN) et user1 (rôle USER). Le LoginController est implémenté comme HttpServlet, avec doGet() pour afficher/quitter et doPost() pour valider les identifiants et créer la session.

## Étape 7 — Implémentation des Filtres

L'AuthFilter est créé pour protéger toutes les ressources. Il vérifie la présence de l'attribut user dans la session et redirige vers /login si absent. Le RoleFilter est créé pour restreindre les opérations d'écriture aux ADMIN. La liste des actions protégées est définie comme une constante Set<String>.

## Étape 8 — Contrôleur des produits

Le ProduitController est implémenté avec doGet() et doPost() qui dispatching via le paramètre action. Chaque action appelle une méthode privée dédiée (handleSearch, handleEdit, handleDelete, handleAdd, handleUpdate). La méthode showList() centralise le forwarding vers la vue index.jsp.

## Étape 9 — Création des vues JSP

Les vues sont placées sous WEB-INF/views pour être inaccessibles directement depuis le navigateur (seuls les contrôleurs peuvent y accéder via forward). La vue index.jsp est développée avec la JSTL pour un rendu conditionnel basé sur les rôles. Les pages d'erreur 404 et error.jsp sont également créées.

## Étape 10 — Configuration du déscripteur web.xml

Le fichier web.xml est configuré pour enregistrer les deux Servlets (LoginController et ProduitController) avec leurs URL patterns, les deux Filtres avec leur ordre d'application, et les pages d'erreur pour les codes 404, 403 et 500. La page d'accueil index.jsp est définie comme welcome-file, et elle effectue immédiatement une redirection vers /login.

## Étape 11 — Initialisation des données et tests

L'AppInitListener est annoté avec @WebListener et implémente contextInitialized() pour insérer les données initiales. Les tests fonctionnels sont réalisés en déployant le WAR sur Apache Tomcat et en vérifiant tous les scénarios :

- Accès sans session : redirection vers login
- Connexion avec admin / admin123 : accès complet (CRUD)
- Connexion avec user1 / pass1 : lecture seule, erreur 403 sur les actions d'écriture
- Ajout, modification et suppression de produits
- Recherche par ID produit
- Déconnexion et nettoyage de session

# 5. Résultats et Tests

| Scénario testé | Résultat attendu | Statut |
|---|---|---|
| Accès sans session | Redirection /login | ✅OK |
| Login admin correct | Session créée, redirection /produits | ✅OK |
| Login user1 correct | Session créée, vue en lecture seule | ✅OK |
| Login invalide | Message d'erreur affiché | ✅OK |
| Ajout produit (ADMIN) | Produit ajouté, liste mise à jour | ✅OK |
| Modification produit (ADMIN) | Produit modifié, liste mise à jour | ✅OK |
| Suppression produit (ADMIN) | Produit supprimé, liste mise à jour | ✅OK |
| Ajout produit (USER) | Erreur 403 Forbidden | ✅OK |
| Recherche par ID | Produit affiché si trouvé | ✅OK |
| Recherche ID inexistant | Liste complète affichée | ✅OK |
| Déconnexion | Session invalide, redirection /login | ✅OK |

# 6. Conclusion

Ce TP a permis de mettre en œuvre une application web Java EE complète en suivant rigoureusement l'architecture MVC2. Les principaux apports de ce travail sont les suivants :

- Maîtrise de l'architecture MVC2 : séparation claire entre le modèle (entités JPA), la vue (JSP/JSTL) et le contrôleur (Servlet)
- Intégration d'Hibernate : utilisation des annotations JPA, gestion des sessions et des transactions avec rollback
- Sécurité multicouche : authentification par session (AuthFilter) et contrôle des accès par rôles (RoleFilter)
- Bonne pratique de conception : patterns DAO, Service et Singleton pour une base de code maintenable et extensible
- Simplicité de déploiement : utilisation de H2 en mémoire et du listener AppInitListener pour une initialisation automatique

L'application est fonctionnelle et couvre tous les cas d'usage attendus. Elle pourrait être enrichie dans une version future par l'ajout d'une base de données persistante (MySQL, PostgreSQL), d'une validation des entrées utilisateur côté serveur, ou encore d'un système de pagination pour la liste des produits.